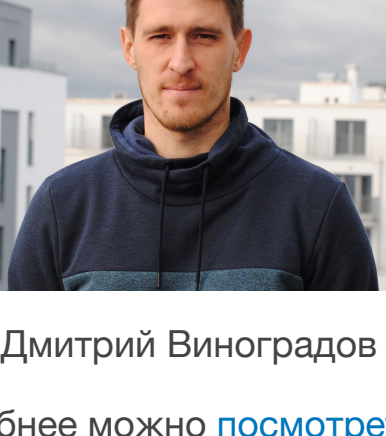


Language



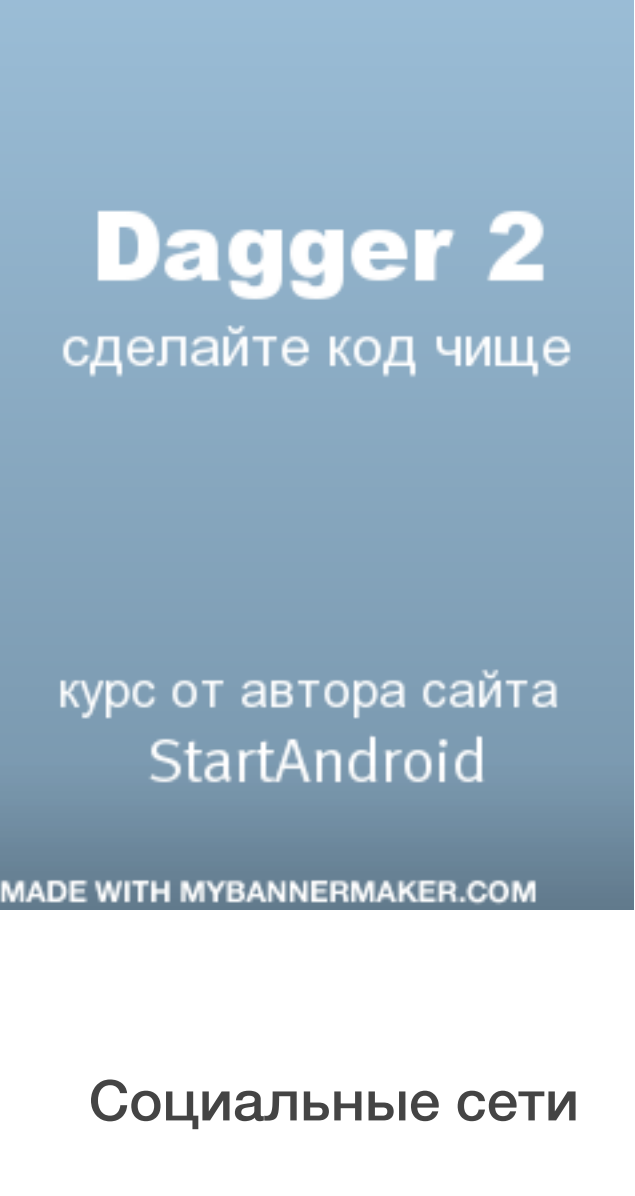
Автор сайта



Дмитрий Виноградов

Подробнее можно [посмотреть](#) или [почитать](#).

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

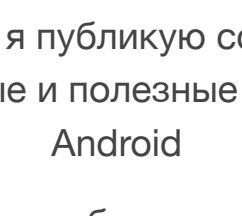


курс от автора сайта

StartAndroid

MADE WITH MYBANNERMAKER.COM

Социальные сети



В [канале](#) я публикую ссылки на интересные и полезные статьи по Android

В [чате](#) можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 086 подписчиков



Подписаться

Поддержка проекта

Яндекс

41001180491924

WebMoney

R248743991365

Z551306702056

Paypal

Урок 2. Preview. Параметры Composable функции

12 декабря 2022

В этом уроке мы используем Preview, чтобы видеть результат работы Composable функции без запуска приложения, рассматриваем некоторые параметры функции Text.

Preview

Одним из плюсов Compose, про который вы наверняка слышали, называют Preview. Это возможность увидеть конечный результат работы Composable функции без запуска приложения. Давайте посмотрим, как это можно сделать

У нас в файле HomeScreen.kt есть функция HomeScreen.

```
@Composable
fun HomeScreen() {
    Text("Home screen")
}
```

Мы хотим прямо в студии видеть, как этот экран будет выглядеть.

Создадим Preview для этой функции в этом же файле:

```
@Preview
@Composable
fun HomeScreenPreview() {
    HomeScreen()
}
```

Это тоже Composable функция, но с дополнительной аннотацией - Preview. Студия автоматически запустит такую функцию, чтобы показать разработчику результат ее работы. А так как внутри этой функции мы вызываем HomeScreen, то студия в итоге вызовет функцию HomeScreen и покажет нам получившийся экран.

Название Preview функции может быть любым. Самый простой шаблон для названия - это имя основной функции + Preview.

Наверняка возникает вопрос: почему нельзя было просто повесить аннотацию Preview на функцию HomeScreen? Зачем создавать обертку?

Потому что у HomeScreen могут быть входные параметры. Например, мы можем передавать ей какой текст, который она должна отображать.

```
@Composable
fun HomeScreen(userName: String) {
    Text("Home screen for $userName")
}
```

Если мы добавим такой функции аннотацию Preview, то студия должна будет запустить ее, чтобы показать нам результат. Но откуда студия знает, какое значение надо в эту функцию передать в качестве userName?

А в Preview обертке мы можем сами указать входные параметры для основной функции:

```
@Preview
@Composable
fun HomeScreenPreview() {
    HomeScreen("test user")
}
```

Студия без проблем запустит HomeScreenPreview, а та уже запустит HomeScreen с нашими параметрами.

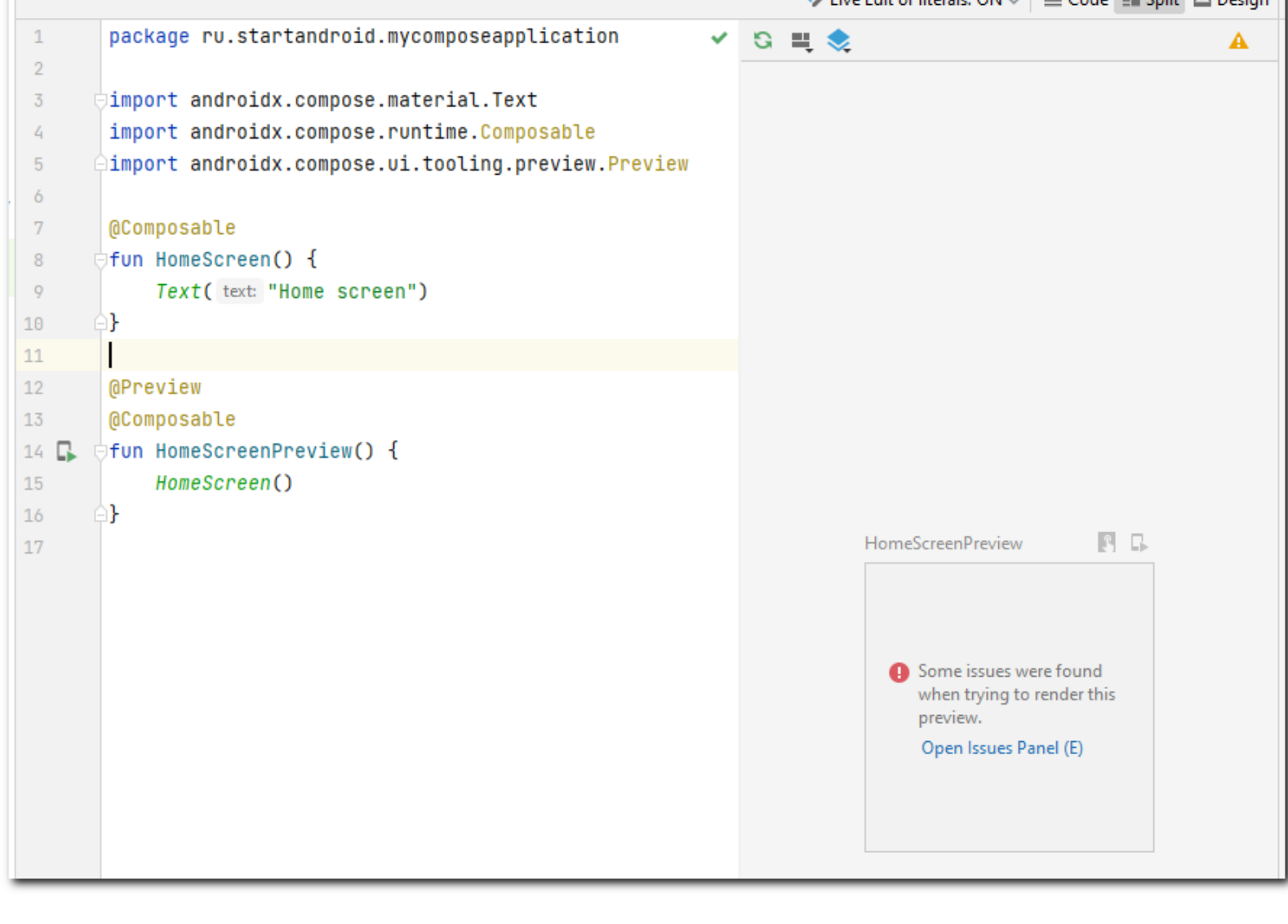
И хотя в нашем примере функция HomeScreen не имеет параметров и ей можно повесить Preview напрямую, но лучше сразу выработать привычку создавать отдельную Preview функцию. Потому что Composable функция без параметров будет встречаться крайне редко.

Итак, у нас есть функция HomeScreen со своим Preview

```
@Composable
fun HomeScreen() {
    Text("Home screen")
}

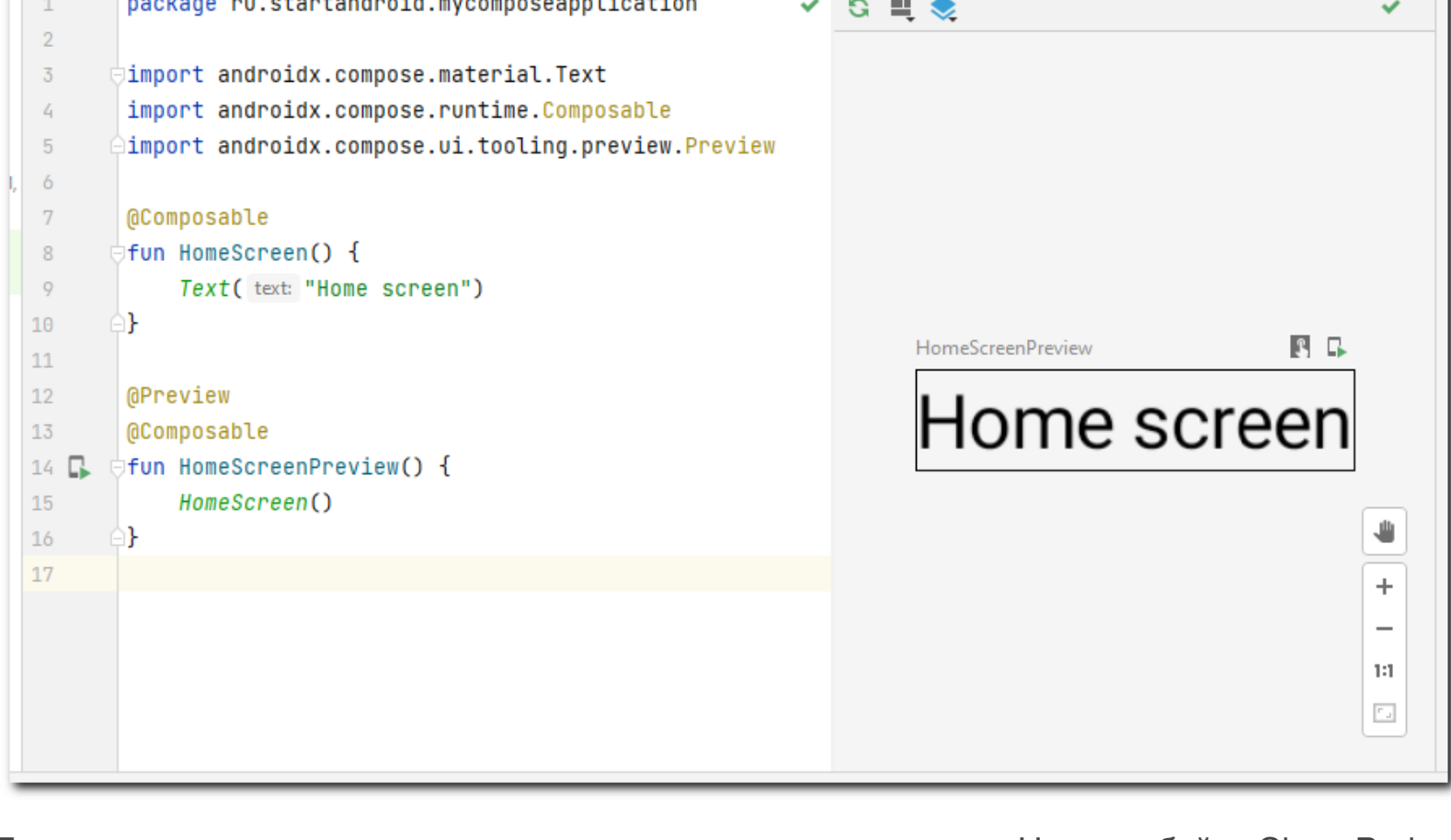
@Preview
@Composable
fun HomeScreenPreview() {
    HomeScreen()
}
```

Чтобы увидеть Preview открываем Split режим:



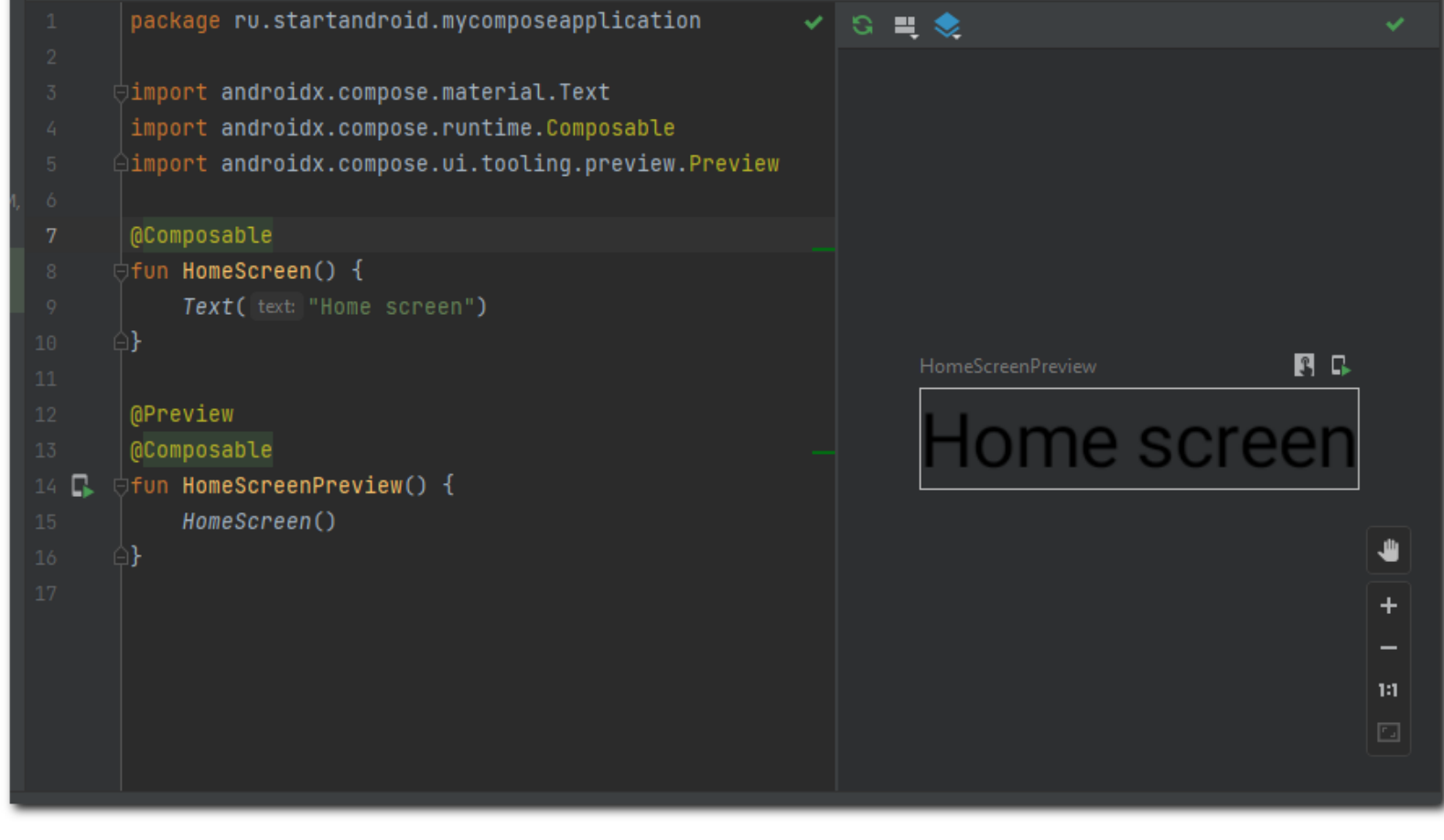
Поначалу студия скорее всего выдаст ошибку.

Чтобы исправить, жмем Ctrl+Shift+F5 (далее Refresh). После некоторых раздумий (Gradle Build Running внизу) студия должна показать результат:



Если вдруг не показывает, то проверьте что все верно в коде. Ну и попробуйте Clean Project, Invalidate Cache и вот это вот все.

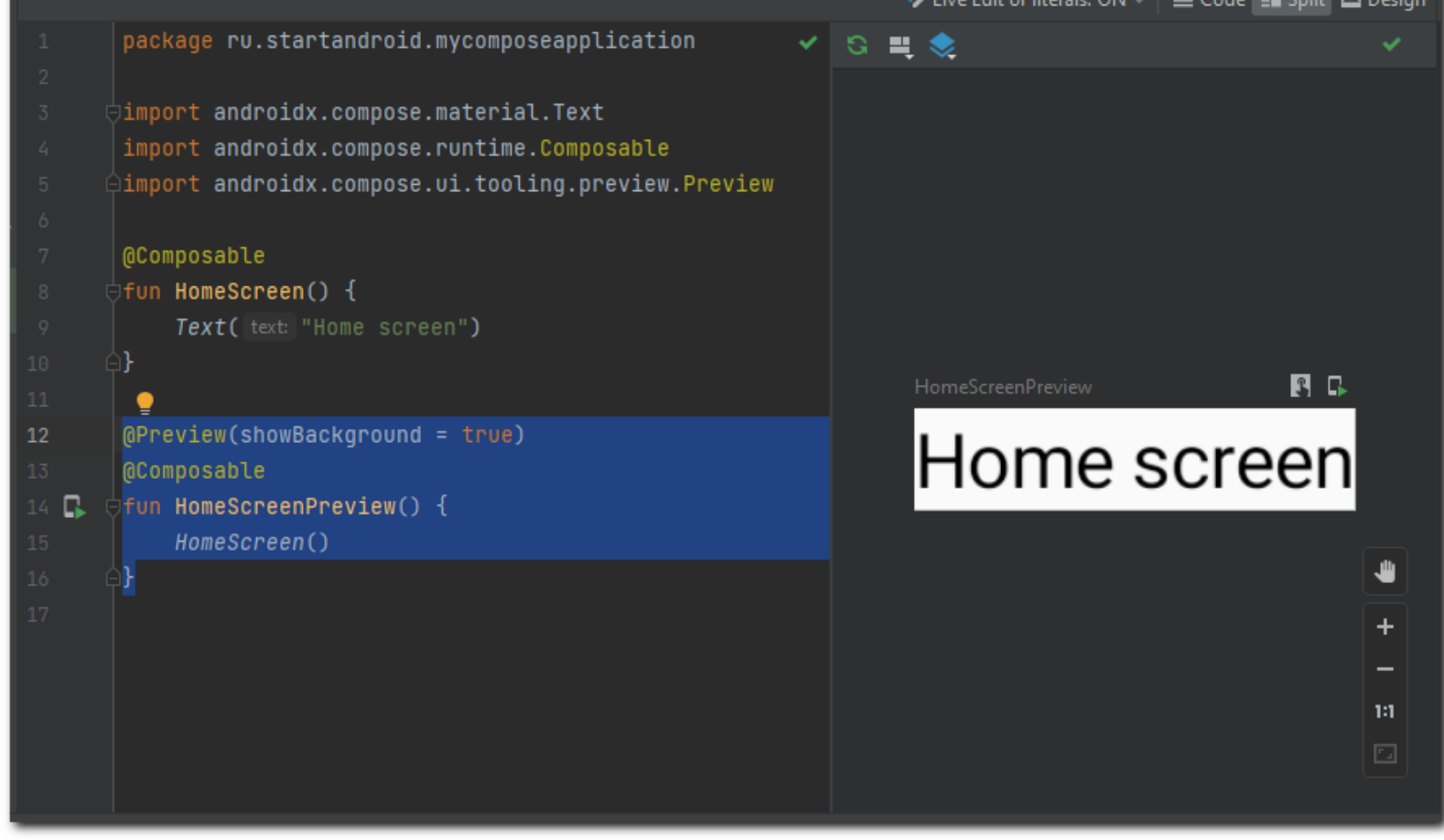
Результат может быть не очень хорошо виден. Особенно если у студии темная тема:



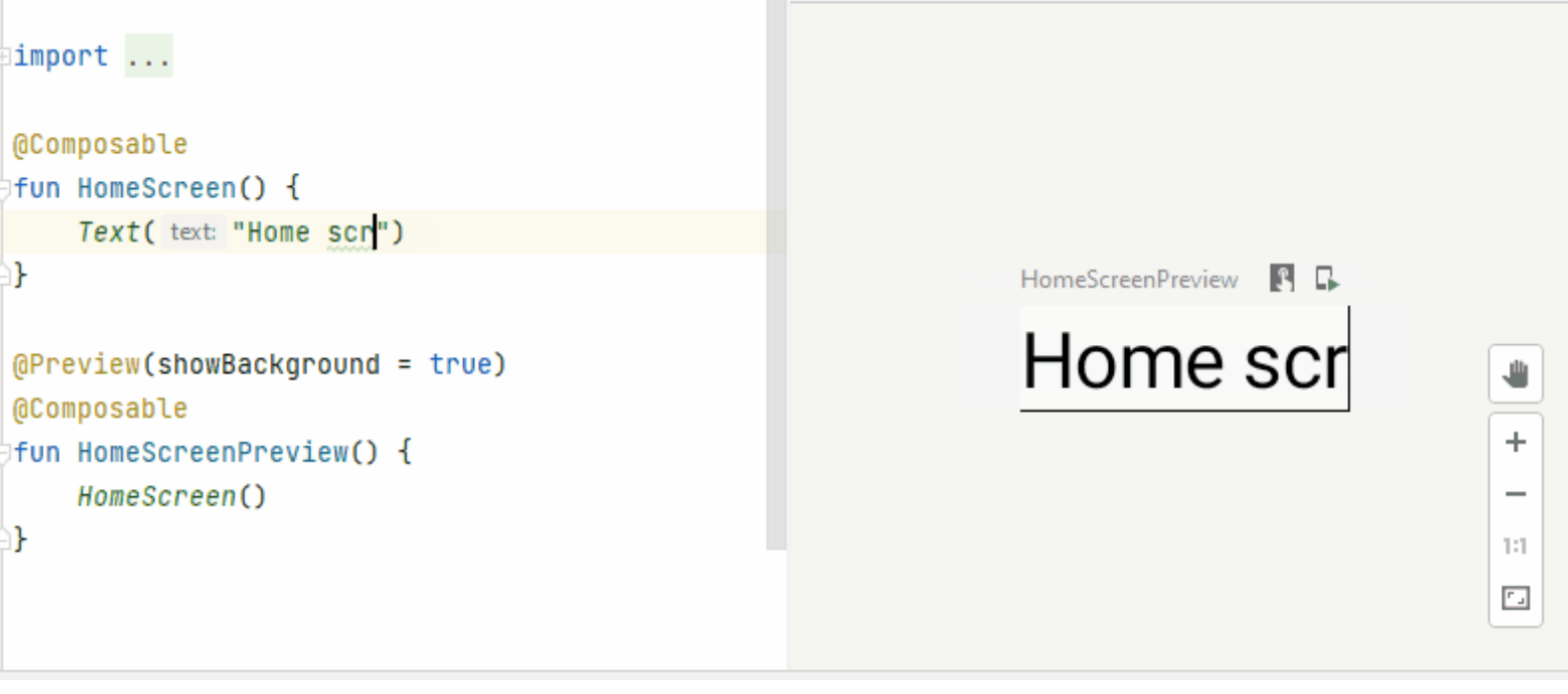
Чтобы это исправить, можно попросить Preview показывать добавит фон.

```
@Preview(showBackground = true)
@Composable
fun HomeScreenPreview() {
    HomeScreen()
}
```

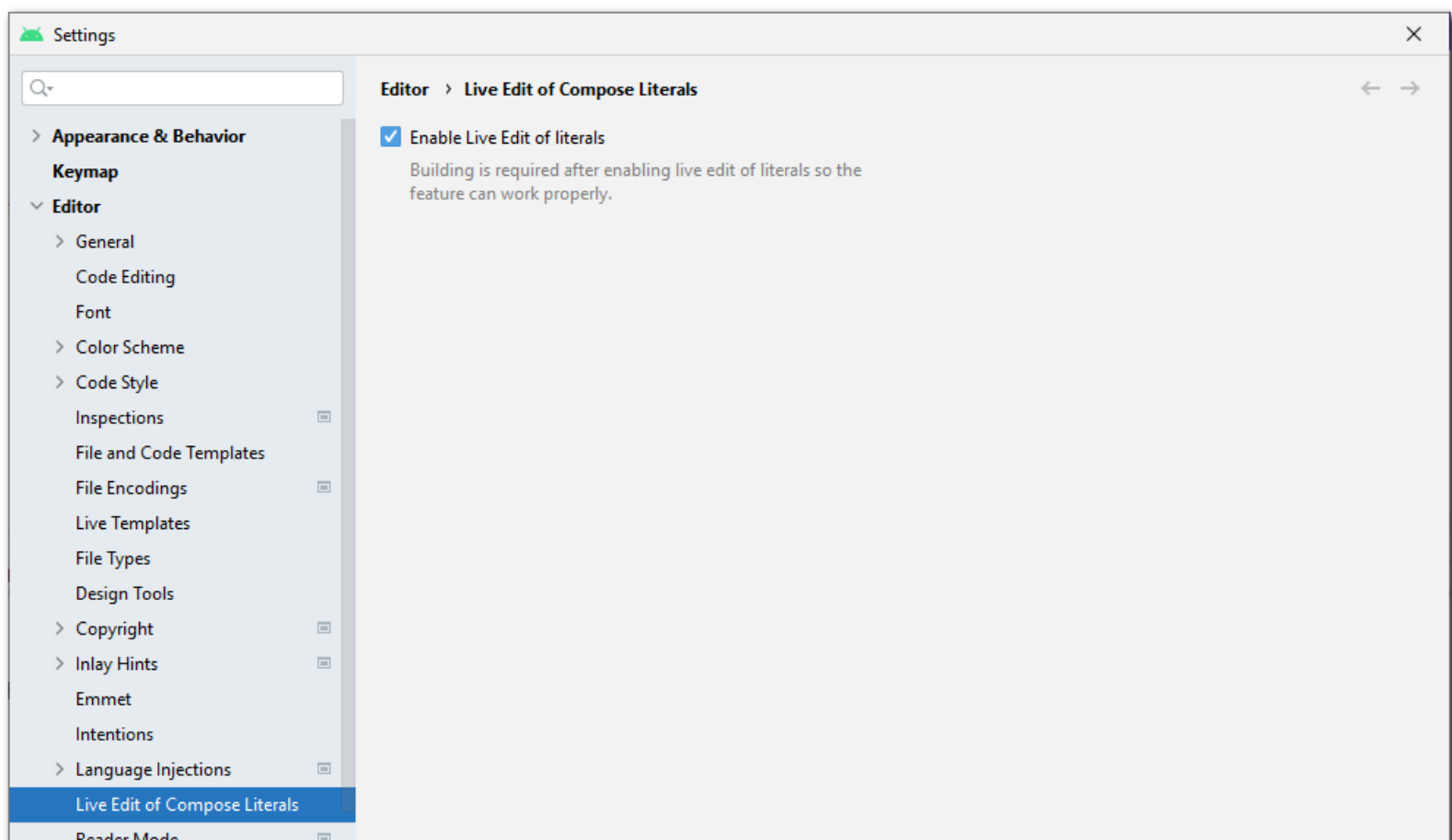
Результат:



Preview умеет подхватывать на лету некоторые изменения, которые мы делаем в функции. Например, мы можем менять текст и Preview сразу это отобразит



Если не меняется, возможно у вас старая версия студии или выключена эта возможность в настройках:



Далее в уроках все скриншоты будут не с Preview, а с экрана эмулятора (если не указано иное).

Параметры Composable функции

Когда мы вызываем Composable функцию, мы передаем ей параметры. С их помощью мы указываем, что именно (и в каком виде) мы хотим видеть на экране. У Composable функции Text мы уже использовали параметр text, чтобы указать, какой текст хотим видеть на экране.

Мы не указывали этот параметр явно, давайте сделаем это:

```
@Composable
fun HomeScreen() {
    Text(text = "Home screen")
}
```

Попробуем использовать другие параметры. Например, укажем размер шрифта и цвет.

Размер шрифта - параметр fontSize:

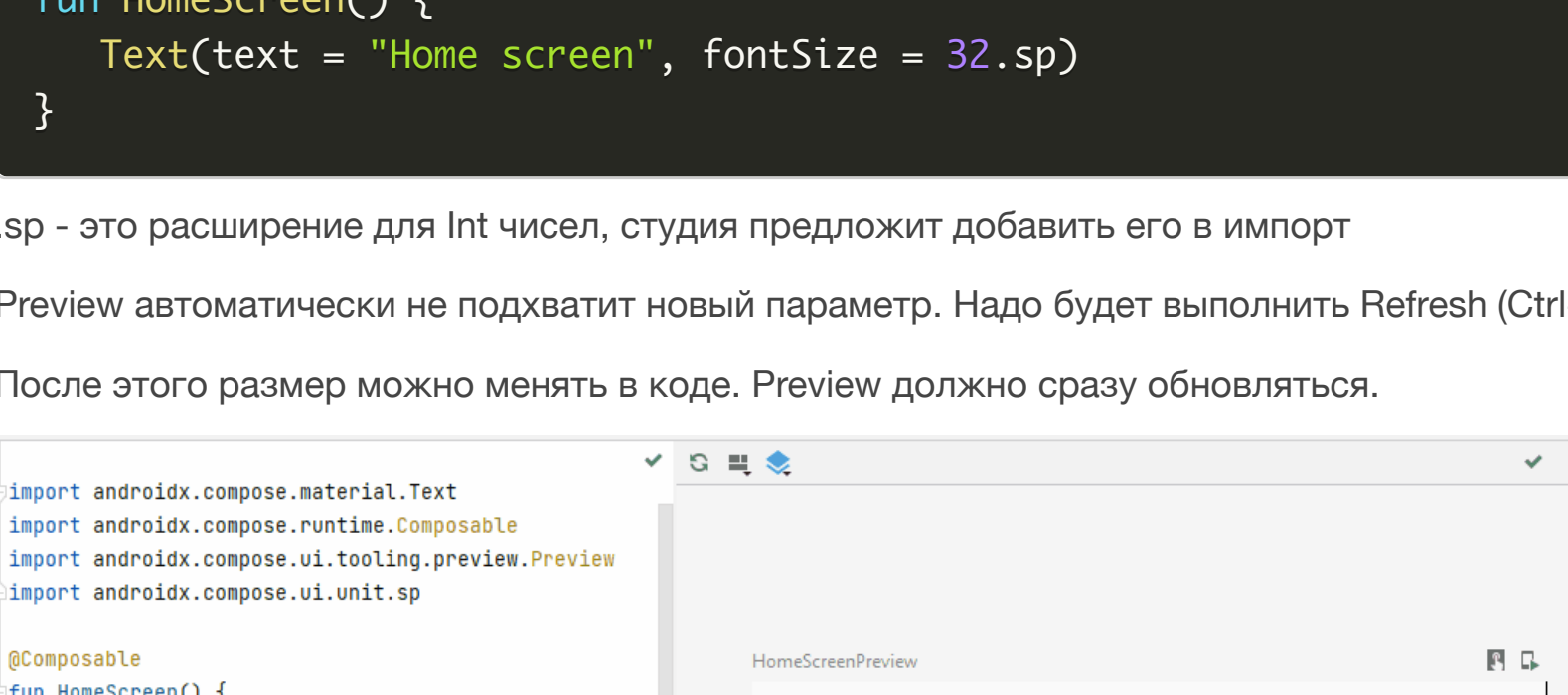
```
import androidx.compose.ui.unit.sp

@Composable
fun HomeScreen() {
    Text(text = "Home screen", fontSize = 32.sp)
}
```

.sp - это расширение для Int чисел, студия предложит добавить его в импорт

Preview автоматически не подхватит новый параметр. Надо будет выполнить Refresh (Ctrl+Shift+F5)

После этого размер можно менять в коде. Preview должно сразу обновляться.

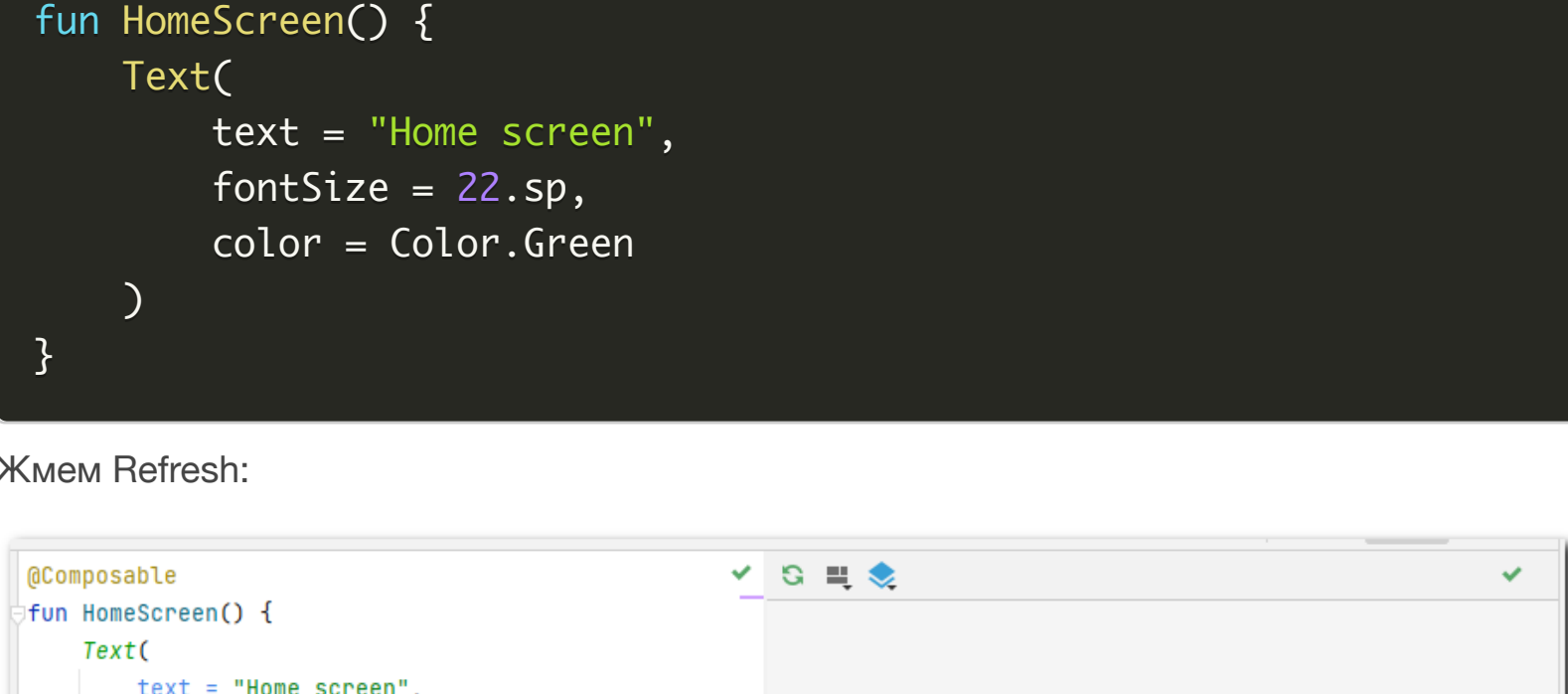


Цвет текста - параметр color:

```
import androidx.compose.ui.graphics.Color

@Composable
fun HomeScreen() {
    Text(
        text = "Home screen",
        fontSize = 22.sp,
        color = Color.Green
    )
}
```

Жмем Refresh:



Изменение цвета в коде не обновляет автоматически, в отличие от размера или текста. Возможно в следующих версиях студии это будет добавлено.

У функции Text есть еще много параметров. Можно открыть исходники Text и посмотреть весь список:

```
@Composable
fun Text(
    text: String,
    modifier: Modifier = Modifier,
    color: Color = Color.Unspecified,
    fontSize: TextUnit = TextUnit.Unspecified,
    fontStyle: FontStyle? = null,
    fontWeight: FontWeight? = null,
    fontFamily: FontFamily? = null,
    letterSpacing: TextUnit = TextUnit.Unspecified,
    textDecoration: TextDecoration? = null,
    textAlign: TextAlign? = null,
    lineHeight: TextUnit = TextUnit.Unspecified,
    overflow: TextOverflow = TextOverflow.Clip,
    softWrap: Boolean = true,
    maxLines: Int = Int.MAX_VALUE,
    onTextLayout: (TextLayoutResult) -> Unit = {},
    style: TextStyle = LocalTextStyle.current
)
```

Многие из них знакомы нам по работе с TextView. Обратите внимание, что тут нет нескольких достаточно важных параметров. Например, нет ничего про размеры элемента: ширину и высоту. Также нет возможности задать фоновый цвет. Для этих атрибутов используется Modifier. Он второй в списке параметров, сразу после text. О нем поговорим в следующем уроке.

Присоединяйтесь к нам в Telegram:

- в [канале StartAndroid](#) публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: [Android](#), [Compose](#), [Kotlin](#), [RxJava](#), [Dagger](#), [Тестирование](#), [Performance](#)

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат [Фудилыня](#)

